MARMARA
UNIVERSITY

Introduction to Embedded Image Processing

EE4065

Homework 4

Mahmut Nedim Göç

150721053

Emre Güner

150722031

Marmara Universty
Faculty of Engineering
Electrical Electronic Engineering

CONTENTS

1. INTRODUCTION

The primary target of the ongoing project is to implement embedded machine learning applications for handwritten digit recognition in an STM32 microcontroller, strictly in accordance with the application examples listed in the course textbook *Embedded Machine Learning with Microcontrollers* (2025). The predominant issue in embedded image processing is the considerable computational overhead in processing the raw image data. To overcome this issue, the project focuses on the Hu moments feature extraction technique and reduces the input dimensionality from the raw 28×28 pixels to a feature vector of only seven dimensions by calculating the first seven invariant moments of the input images.

By doing this, the project reduces the input image size to a feature vector with a substantially lower number of dimensions, significantly reducing the amount of memory needed and the computational overhead involved in the process, making the project ideal for implementation in an energy-constrained environment like the STM32 microcontroller.

For this efficient manner of feature extraction to be implemented, the project incorporates two different models of artificial intelligence based on the textbook described above. The first application (Chapter 10: Section 10.9) involves the implementation of a Single Neuron model with a Sigmoid activation function to accomplish a binary classification task to distinguish the digit ’0’ from the rest of the digits. The model will serve as an introductory example for the learning pipeline process implemented in the project. The second application (Chapter 11: Section 11.8) involves the implementation of a Multi-Layer Perceptron (MLP) model with hidden layers and a ReLU activation function to accomplish the classification of digits ranging from "0" to "9." The implementation process involves a combination of strategies where the models will first undergo offline training in Python (TensorFlow/Keras) to calculate the optimal weights and biases, which will then be saved into a C header file. The final inference

engine process, involving the complete calculation of the mathematical process of the Hu moments, normalization processes, and propagation processes in the artificial intelligence model, will be implemented in its entirety in C in the STM32CubeIDE environment.

2. PROBLEMS

Implement the below end of chapter applications from the book below. Please use the mentioned offline data sets there.

C. Ünsalan, B. Höke, and E. Atmaca, Embedded Machine Learning with Microcontrollers: Applications on STM32 Boards, Springer Nature, ISBN: 978-3031709111, 2025

(1) **Section 10.9 Application: Handwritten Digit Recognition from Digital Images**

(2) **Section 11.8 Application: Handwritten Digit Recognition from Digital Images**

3. RESULTS

3.1. METHODOLOGY: BINARY DIGIT RECOGNITION SYSTEM

In this study, a lightweight machine learning pipeline was developed to distinguish the digit “0” from other digits (“1-9”) using the MNIST dataset. The system architecture consists of two main stages: offline training using Python/TensorFlow and real-time inference deployed on an STM32 microcontroller.

3.1.1. Feature Extraction and Preprocessing

To enable efficient processing on a resource-constrained embedded device, we utilized feature extraction rather than processing raw pixel data. For each image in the MNIST dataset, **Hu Invariant Moments** were calculated using the OpenCV library. Hu Moments provide a set of seven invariants that are robust to translation, scale, and rotation, making them ideal descriptors for handwritten digits.

The extracted feature vectors X were standardized to ensure faster convergence during training. The standardization process is defined as:

$$x_i' = \frac{x_i - \mu_i}{\sigma_i} \tag{3.1}$$

where x_i is the raw feature, μ_i is the mean, and σ_i is the standard deviation of the training set. These statistical parameters (μ and σ) were recorded for later use in the embedded implementation.

3.1.2. Model Architecture and Training

A single-neuron classifier (Perceptron) was designed using the TensorFlow Keras API. The model consists of a dense layer with a single unit and a **Sigmoid** activation function, which outputs a probability score between 0 and 1.

Since the dataset is imbalanced (the class “0” represents approximately 10% of the data, while “non-zero” represents 90%), a class weighting strategy was implemented. We assigned a weight of 8 to class “0” and 1 to class “1” to penalize misclassifications of the minority class more heavily. The model was trained using the **Adam optimizer** and **Binary Cross-Entropy loss** for 50 epochs.

The performance of the trained model was evaluated using a confusion matrix, as shown in Figure 3.1.

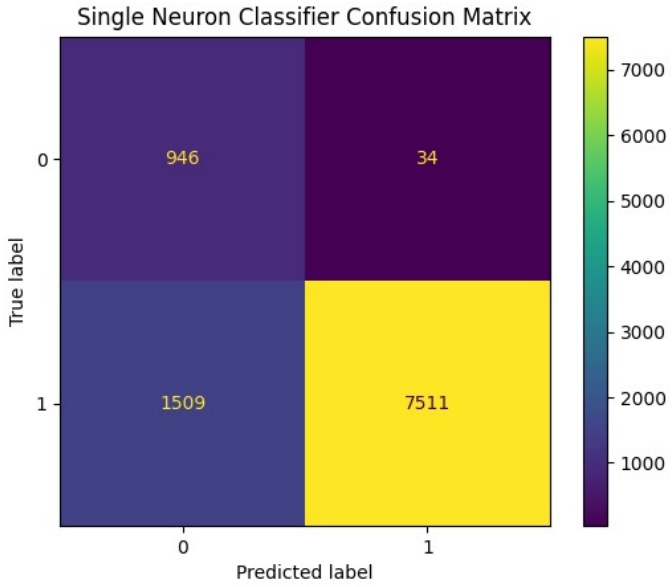

Figure 3.1: Confusion Matrix of the Single Neuron Classifier

3.1.3. Embedded Implementation on STM32

The core contribution of this work is the deployment of the trained model onto an STM32 microcontroller. Since standard Python libraries cannot run directly on bare-metal microcontrollers, we implemented a “Weight Export” mechanism.

- **Parameter Extraction:** After training, the optimized weights (W) and bias (b), along with the normalization parameters (Mean μ and Std σ), were extracted from the model.
- **Header File Generation:** A Python script formatted these parameters into C-style arrays and generated a header file (`model_data_q1.h`). This allowed the STM32 to store the parameters in flash memory.

- **Inference Engine:** On the STM32, a C function replicated the forward propagation logic. The microcontroller performs the dot product and applies the sigmoid function:

$$z = \left(\sum_{i=1}^{7} w_i \cdot x'_i \right) + b \tag{3.2}$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{3.3}$$

If the output $\hat{y}$ exceeds 0.5, the input is classified as “not zero”. The successful execution on the STM32 is demonstrated in Figure 3.3.

Validation with a Test Sample

To validate the accuracy of the deployed model on the microcontroller, a specific test sample representing the digit “7” was selected from the MNIST test dataset. This sample image is presented in Figure 3.2.

The Hu moment features corresponding to this specific “7” were extracted and fed into the STM32 inference engine as the input vector. Since “7” belongs to the “not-zero” class, the expected output from the sigmoid function is a probability close to 1.0. The subsequent classification result stored in the microcontroller’s memory matches this expectation, as detailed in Figure 3.3.

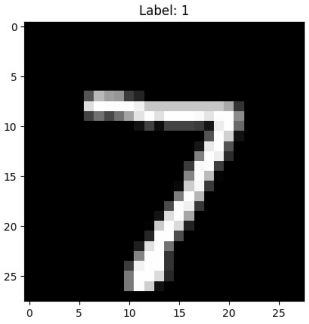

Figure 3.2: The sample image of digit ’7’ used for embedded testing.

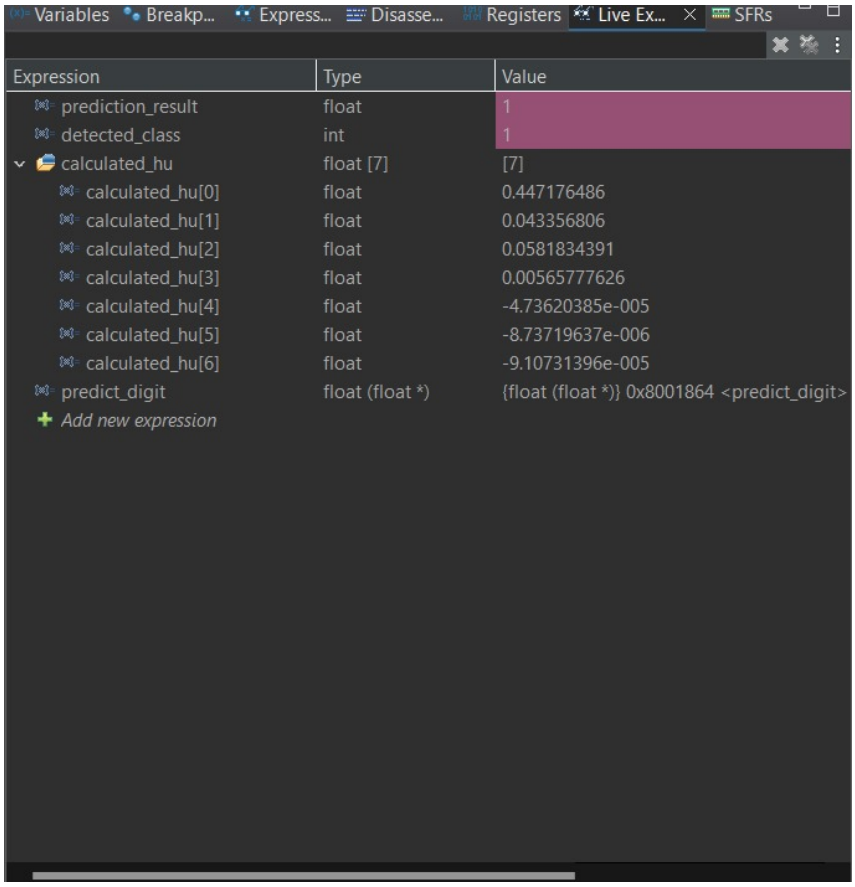

Figure 3.3: STM32 Memory View showing the inference result for the digit ’7’.

3.2. MULTI-CLASS HANDWRITTEN DIGIT RECOGNITION USING MLP

Building upon the binary classification task, this section extends the application to recognize all ten digits (0-9) from the MNIST dataset. Since a single neuron is insufficient for separating multiple non-linearly separable classes, a Multilayer Perceptron (MLP) neural network architecture was employed.

3.2.1. Network Architecture and Training Strategy

The proposed neural network consists of an input layer, two hidden layers, and an output layer. The feature extraction process remains consistent with the previous section, utilizing the 7 Hu Moments as the input vector.

The network topology is defined as follows:

- **Input Layer:** 7 neurons corresponding to the Hu Moment features.

- **Hidden Layers:** Two dense layers, each containing 100 neurons. The **ReLU** (Rectified Linear Unit) activation function is used in these layers to introduce non-linearity, enabling the model to learn complex patterns.
- **Output Layer:** 10 neurons corresponding to the digits 0 through 9. The **Softmax** activation function is applied to convert the raw output logits into a probability distribution over the predicted classes.

Training was performed using the **Adam optimizer** with a learning rate of $1e^{-4}$. The **Sparse Categorical Cross-Entropy** loss function was selected as it allows for the direct use of integer labels without requiring one-hot encoding. To ensure optimal generalization and prevent overfitting, two critical callbacks were implemented:

1. **ModelCheckpoint:** Saves the model weights only when the validation performance improves.
2. **EarlyStopping:** Monitors the loss metric and halts the training process if no improvement is observed for a specified patience period (5 epochs).

The classification performance of the trained MLP model is visualized in the Confusion Matrix shown in Figure 3.4.

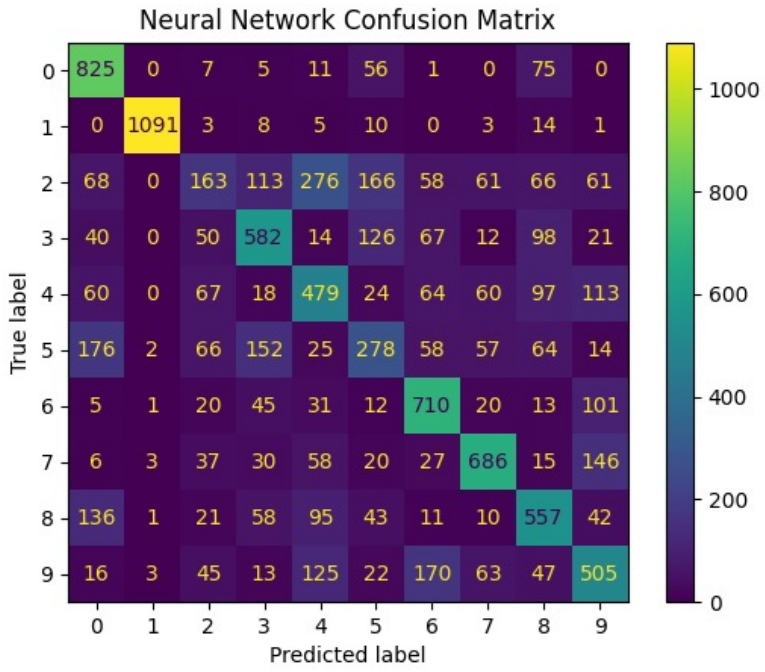

Figure 3.4: Confusion Matrix of the Multilayer Perceptron (0-9 Classification)

3.2.2. Embedded Implementation on STM32

Deploying a multilayer network on the STM32 microcontroller requires a more complex inference engine compared to the single neuron approach. The weights and biases for all three layers were exported from the Python environment into a C header file (`model_data_q2.h`).

The inference process on the STM32 involves sequential matrix multiplications and activation function calls. The forward propagation logic implemented in C is described mathematically as:

$$h_1 = \text{ReLU}(W_1 \cdot x + b_1) \tag{3.4}$$

$$h_2 = \text{ReLU}(W_2 \cdot h_1 + b_2) \tag{3.5}$$

$$y_{out} = \text{Softmax}(W_3 \cdot h_2 + b_3) \tag{3.6}$$

Where x is the input feature vector, and W_i, b_i are the weights and biases for layer i. On the microcontroller, the final prediction is obtained by finding the index of the neuron with the highest probability value (argmax) in the output layer.

The implemented C code was successfully tested on the STM32 development board. Figure 3.5 illustrates the memory view of the microcontroller during runtime, confirming that the inference engine correctly predicts the input digit.

Validation with Test Sample

Consistent with the validation approach used in the binary classification task, the digit “7” was again utilized as a test sample to verify the accuracy of the multi-class model on the microcontroller. The extracted features of the test image were processed by the inference engine on the STM32.

Since the output layer uses the Softmax activation function, the model produces a probability distribution across ten classes (indices 0 to 9). As expected, the embedded implementation correctly identified the input by assigning the highest probability value to the neuron corresponding to class “7”. This successful prediction confirms the model’s functionality on the target hardware, as evidenced by the memory observation in Figure 3.5.

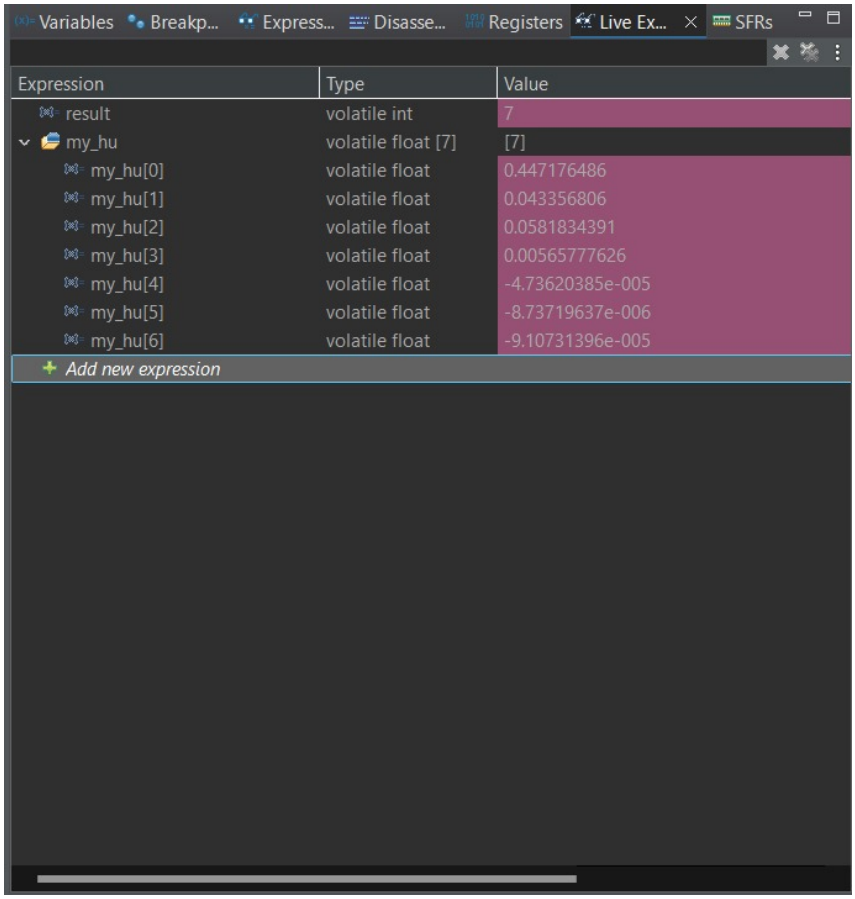

Figure 3.5: STM32 Memory View showing the classification result for the multi-class MLP.

4. CONCLUSION

For the purpose of this scenario, two different machine learning architectures were designed and evaluated successfully on an STM32 microcontroller, thus proving the viability of artificial intelligence in embedded systems. The most crucial aspect of the design process was the utilization of Hu moments in the feature extraction phase, which successfully reduced the high-dimensional raw image input (in terms of pixels, ranging to a total of 784) to a low-dimensional feature vector consisting of seven elements. This was a crucial factor in making the AI inference feasible in the resource-limited environment of the microcontroller.

The methodology has created an effective way to embed machine learning into a project: the model was developed in a higher-level environment (Python and TensorFlow) and the resulting optimal parameters were implemented in an embedded environment (C and STM32 microcontroller). The Single Neuron classifier (binary classification) and the Multi-Layer Perceptron (digit classification) worked properly in the embeddable environment. Final testing, involving a comparison of the STM32 outputs in the debug environment and the model outputs in Python, has proved that the embedded inference engine runs properly and accurately in a real-time environment. The results prove that complex pattern recognition problems can be solved in a typical microcontroller if proper feature extraction and optimization of the algorithm are applied.

REFERENCES

(**1**) Embedded Machine Learning with Microcontrollers Applications on STM32 Development Boards, Cem Ünsalan, Berkan Höke and Eren Atmaca

(**2**) Embedded System Design with ARM Cortex-Microcontrollers: Applications with C, C++ and Pyhton, Cem Ünsalan, Hüseyin Deniz Gürhan and Mehmet Erkin